KAVINMITHA.S
192511285

CO2-AT3 SCENARIO BASED TASK

CSA1402 Compiler Design

1. Scenario : Parsing Table Conflict in LL(1), while designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This lead to confusion. identify the reason and explain its effect on deterministic parsing.

* LL(1) parser uses FIRST and FOLLOW sets to build the parsing table.
* A conflict occurs when a table cell contains more than one production.
* This happens due to left recursion, common prefixes (left factoring) or ambiguous grammar.
* The parser cannot decide which production to apply
* This makes parsing non-deterministic.
* Decision : Remove left recursion, apply left factoring and ensure no FIRST/FOLLOW overlap to obtain an LL(1) grammar.

2. Scenario : Incorrect Parse tree Generation A Syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators.

* The Syntax analyzer builds a parse tree using grammar rules
* Incorrect grouping changes the expression evaluation.
* The issue is caused by missing associativity rules.
* Multiple parse trees may be generated for the same expression.
* Addition and Subtraction should follow left associativity.
* Decision : Modify the grammar to enforce left associativity for correct parsing.

Scenario : Invalid Expression Handling , A Syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer.

* The expression a + * b has valid tokens by an invalid order.
* After '+', the grammar expects an operand, not another operator.
* Predictive parsing reports an error when no valid production exists.
* Shift-reduce parsing reports an error when no valid action available. Parsing stops without recovery
* Decision : Use panic-mode or phrase-level error recovery with meaningfuly error messages.

4. Ambiguity in Conditional Statements

* The grammar is ambiguous because of the dangling else problem.
* The parser cannot decide which if matches the else.

* This creates parsing conflicts and multiple phase trees.
* Deterministic parsing becomes difficult.
* Rewrite the grammar using matched & unmatched statements.
* Decision: Associate else with the nearest unmatched if.

5. Operator Precedence Conflict

* The grammar is ambiguous because precedence is not defined.
* The expression id + id * id can produce different parse trees.
* Multiplication should follow ~~left~~ higher precedence than addition.
* Operators should follow left associativity.
* Ambiguity leads to incorrect evaluation.
* Decision: Re write the grammar using seperate non-terminals (E, T, F) or use precedence declarations in the parser.